BCA Semester 1 Examination 2025

Model Solutions: 1BCA3 - Programming Logic & Design Techniques

Session	Paper Code	Time	Full Marks	Pass Marks
2024-2027	111603	3 Hours	100	40

Instructions: Answer any five questions (equal marks). Solutions for all ten questions are given below for practice. Write answers in your own words in the exam.

Q1. What is time complexity? Discuss worst case, average case, best case and different notations.

Time complexity

Time complexity of an algorithm is the amount of time it needs to complete as a function of the input size n. It is measured by counting the number of basic operations (comparisons, assignments), not in seconds, so that it is independent of the machine and language.

Best, average and worst case

Case	Meaning	Example: linear search in n elements
Best case	Minimum time over all inputs of size n	Key is the first element: 1 comparison, O(1)
Average case	Expected time over all inputs	Key is in the middle: about n/2 comparisons, O(n)
Worst case	Maximum time over all inputs (guarantee)	Key is last or absent: n comparisons, O(n)

Asymptotic notations

Notation	Name	Definition	Meaning
O(g(n))	Big-O	f(n) <= c*g(n) for n >= n0	Upper bound (worst case)
Omega(g(n))	Big-Omega	f(n) >= c*g(n) for n >= n0	Lower bound (best case)
Theta(g(n))	Big-Theta	c1*g(n) <= f(n) <= c2*g(n)	Tight bound
o(g(n)), w(g(n))	Little-o, little-omega	Strict versions of O and Omega	Strictly smaller / larger growth

Example: f(n) = 3n + 2. Since 3n + 2 <= 4n for n >= 2, f(n) = O(n); also f(n) >= 3n so f(n) = Omega(n); hence f(n) = Theta(n). Common orders: O(1) < O(log n) < O(n) < O(n log n) < O(n^2) < O(2^n).

Q2. How do features of an algorithm help us in solving a problem in the best way?

An algorithm is a finite sequence of well-defined steps to solve a problem. Its essential features and their help are:

Feature	Meaning	How it helps in solving the problem
Input	Zero or more well-defined inputs	Clearly states the data required, so the problem is properly defined.
Output	At least one result	Ensures the algorithm produces the answer that is needed.
Definiteness	Every step is precise and unambiguous	Anyone (or any computer) following it gets the same result; no confusion.

Feature	Meaning	How it helps in solving the problem
Finiteness	Ends after a finite number of steps	Guarantees that the solution is reached and the program does not run forever.
Effectiveness	Every step is simple and can be done in finite time	Makes the algorithm practical to implement and run.
Correctness	Gives the right output for all valid inputs	Gives a reliable solution.
Efficiency	Uses little time and memory	Chooses the best of several solutions.
Generality / independence	Works for a class of problems, independent of a language	Can be reused and coded in any language.

Because an algorithm breaks a problem into small ordered steps, it makes the logic easy to understand, test, debug and improve before coding, and it allows different solutions to be compared by their time and space complexity.

Q3. What is space complexity? How is it measured?

Space complexity is the total amount of memory an algorithm needs to run to completion, as a function of the input size n.

Measurement

$S(P) = C + S_p(I)$

- **C - fixed part:** memory independent of the input: program code, simple variables, constants.
- **$S_p(I)$ - variable part:** memory that depends on the input: arrays, dynamically allocated memory, and the recursion stack.

It is expressed with asymptotic notation, e.g. O(1) or O(n), counting the number of memory words used by the data.

Algorithm	Extra space used	Space complexity
Sum of n numbers using one variable (loop)	sum, i	O(1)
Copying an array of n elements	New array of size n	O(n)
Recursive factorial of n	n stack frames	O(n)
Merge sort	Temporary array	O(n)

Example: for `sum = 0; for(i=0;i<n;i++) sum += a[i];` the array a needs n words and sum, i, n need 3 words, so S = n + 3, i.e. O(n) (O(1) if only the extra space is counted).

Q4. What is inheritance? Discuss features and types of inheritance.

Inheritance

Inheritance is an object-oriented feature in which a new class (derived / child / sub class) acquires the properties (data members) and behaviour (methods) of an existing class (base / parent / super class). It models an 'is-a' relationship, e.g. Car is a Vehicle.

Features / advantages

- **Code reusability:** existing code is reused instead of rewritten.
- **Extensibility:** the derived class can add new members or override base class methods.

- **Reduced redundancy and easy maintenance:** a change in the base class reflects in all derived classes.
- **Supports polymorphism** and a natural hierarchical classification.

Types of inheritance

Type	Description	Example
Single	One derived class from one base class.	Vehicle -> Car
Multilevel	A chain: derived class becomes base for another.	Person -> Employee -> Manager
Hierarchical	Several classes derive from one base class.	Shape -> Circle, Rectangle
Multiple	One class derives from two or more base classes.	Teacher + Researcher -> Professor
Hybrid	Combination of two or more types above.	Multilevel + Multiple

```
class Vehicle { public: void start() { } };       // base class
class Car : public Vehicle { public: void ac() { } };   // single inheritance
// Car object can call both start() and ac()
```

Q5. Explain procedural and recursive programming.

Procedural programming

A program is written as a set of procedures (functions) that are called in order. Control is by sequence, selection and loops; the emphasis is on *how* the task is done. Data and procedures are separate. Languages: C, Pascal, FORTRAN. Benefits: modularity, reuse of procedures, simple top-down design.

Recursive programming

A function calls itself with a smaller input until a **base case** is reached. It has a base case (stops) and a recursive case (moves towards the base case). Each call is stored on the stack. Suited to factorial, Fibonacci, tree traversal, Tower of Hanoi.

```
int fact(int n)
{
    if (n <= 1)  return 1;          /* base case */
    return n * fact(n - 1);         /* recursive case */
}
```

Basis	Procedural (iterative)	Recursive
Repetition	Loops (for, while)	Self function calls
Stops when	Loop condition is false	Base case is reached
Memory / speed	Less memory, faster	More stack memory, slower
Code	Longer	Short, elegant

Q6. Explain nesting of IF-ELSE statement using an algorithm.

When an if-else statement is placed inside the if part or else part of another if-else, it is called **nested if-else**. It is used when a decision depends on several conditions in sequence. The inner condition is tested only if the outer condition is satisfied.

```
IF condition1 THEN
    IF condition2 THEN
        statement A
```

```
    ELSE
        statement B
    ENDIF
ELSE
    statement C
ENDIF
```

Algorithm: find the largest of three numbers A, B, C

1 Start.

2 Read A, B, C.

3 If A > B then: if A > C then Largest = A, else Largest = C.

4 Else (A <= B): if B > C then Largest = B, else Largest = C.

5 Print Largest.

6 Stop.

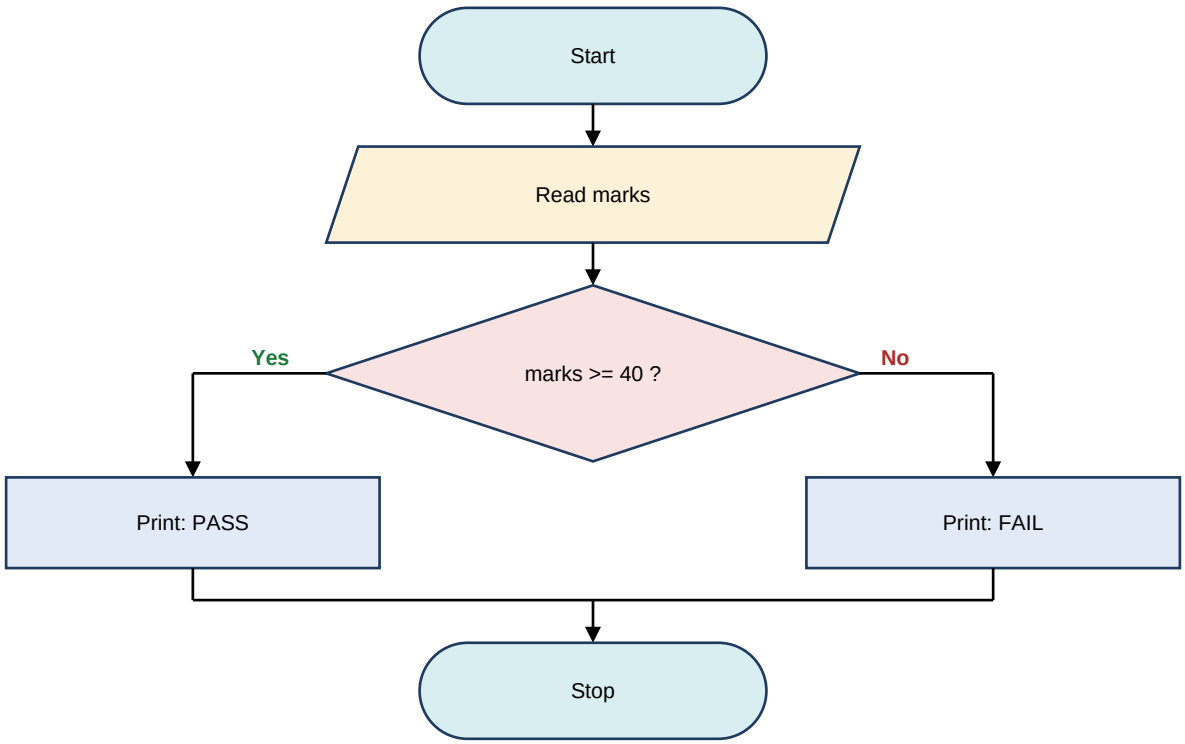
Trace: A = 12, B = 30, C = 21: A > B is false, so test B > C: 30 > 21 is true, so Largest = 30.

Algorithm: grade of a student (else-if ladder form)

```
READ marks
IF marks >= 75 THEN grade = 'A'
ELSE IF marks >= 60 THEN grade = 'B'
ELSE IF marks >= 40 THEN grade = 'C'
ELSE grade = 'Fail'
PRINT grade
```

Q7. Design a flowchart for representing decision-making statements. Discuss different symbols used in flowchart.

A decision-making statement chooses one of two paths according to a condition. It is drawn with the **diamond** decision symbol having two exits, Yes (True) and No (False). Flowchart for the if-else decision: *check whether a student has passed (marks >= 40)*.



Symbols used in flowchart

Symbol	Name	Use
Oval	Terminator	Start / Stop
Parallelogram	Input / Output	Read and print data
Rectangle	Process	Calculation, assignment
Diamond	Decision	Condition with Yes/No outputs (if, loop test)
Arrow	Flow line	Direction of flow
Circle	Connector	Connects flowchart parts
Double-sided rectangle	Predefined process	Function / subroutine call

Q8. What is pseudo code? Write a pseudo code for printing table of 5 (five).

Pseudo code

Pseudo code is an informal description of an algorithm in simple English mixed with programming keywords (READ, PRINT, IF, WHILE, FOR). It cannot be executed but is easy to read and easy to convert into any programming language; it helps in planning logic before coding. It has no strict syntax.

Pseudo code for the table of 5

```
BEGIN
    SET n = 5
    FOR i = 1 TO 10 DO
        product = n * i
        PRINT n, " x ", i, " = ", product
    END FOR
END
```

Output

```
5 x 1 = 5
5 x 2 = 10
5 x 3 = 15
...
5 x 10 = 50
```

Equivalent WHILE form: set i = 1; WHILE i <= 10: print 5 x i; i = i + 1.

Q9. Write short notes on any two of the following: (a) Variable (b) Constant (c) Data type (d) Encapsulation

(a) Variable

A variable is a named memory location used to store data that may change during execution of a program. It has a name (identifier), a data type and a value. It must be declared before use. Example in C: `int age = 18; age = 19;`

(b) Constant

A constant is a fixed value that does not change during execution. Types: numeric (100, 3.14), character ('A') and string ("BCA"). Named constants are created with `const float PI = 3.14;` or `#define MAX 100`.

(c) Data type

A data type defines the kind of value a variable can hold, the memory it occupies and the operations allowed on it.

Category	Types	Typical size
Primitive (basic)	int, float, double, char	4, 4, 8, 1 bytes
Derived	array, pointer, function	depends
User defined	struct, union, enum, class	depends

(d) Encapsulation

Encapsulation is the binding of data and the methods that operate on it into a single unit (class), with data hiding through access specifiers (private, protected, public). Outside code can reach private data only through public methods. Benefits: security, modularity and easier maintenance. Example: a BankAccount class keeps the balance private and allows changes only via deposit() and withdraw().

Q10. What is iteration and selection control structure?

Control structures decide the order of execution of statements. The three basic ones are sequence, selection and iteration.

Selection control structure

Selection (decision / branching) executes one block of statements out of several depending on a condition. Forms: if, if-else, nested if-else, else-if ladder and switch-case.

```
if (marks >= 40)
    printf("Pass");
else
    printf("Fail");
```

Iteration control structure

Iteration (looping / repetition) repeats a block of statements as long as a condition is true. Forms: **for** (known number of times), **while** (condition tested first) and **do-while** (body runs at least once).

```
for (i = 1; i <= 5; i++)
    printf("%d ", i);          /* prints 1 2 3 4 5 */
```

Basis	Selection	Iteration
Purpose	Choose a path	Repeat a block
Executes	Once (one of the branches)	Many times
Statements	if, if-else, switch	for, while, do-while
Flowchart	Diamond with two or more exits	Diamond with a flow line returning back